Projet informatique

Gestion des colis et optimisation du chargement d'un camion

Rapport technique

Auteur : [Prénom NOM]

Langage : Python 3

Date : Octobre 2026

Dépôt GitHub : https://github.com/BilalBSK/gestion-colis

Sommaire

1. Introduction et choix du langage
2. Organisation du projet
3. Modélisation des données
4. Gestion des colis : ajout, affichage, recherche
5. Tri des colis
6. Interface utilisateur
7. Passage à l'échelle
8. Optimisation du chargement du camion
9. Prise en compte de la priorité
10. Étude des performances
11. Synthèse des choix algorithmiques majeurs
12. Questions de réflexion
13. Conclusion

1. Introduction et choix du langage

1.1 Contexte

Une entreprise de livraison prépare chaque jour le chargement de ses camions à partir d'une liste de colis. Chaque colis est décrit par un identifiant, un poids, une valeur, une priorité et une destination.

Le programme réalisé doit répondre à deux besoins :

- **gérer les colis** : ajouter, afficher, rechercher et trier une collection pouvant contenir jusqu'à 100 000 colis, voire davantage ;
- **préparer un camion** : choisir les colis à charger sans dépasser la capacité du camion, en transportant la plus grande valeur possible et en favorisant les colis prioritaires.

1.2 Choix du langage : Python

Le projet est réalisé en **Python 3**, avec la bibliothèque standard uniquement. Ce choix repose sur trois raisons :

- **Lisibilité.** Le code Python reste proche du pseudo-code. L'évaluation portant d'abord sur les choix algorithmiques, on peut se concentrer sur les algorithmes plutôt que sur la gestion manuelle de la mémoire (allocation, libération, tailles de tableaux).
- **Outils intégrés.** La bibliothèque standard fournit tout le nécessaire pour tester (`unittest`), mesurer les temps d'exécution (`time`) et la mémoire (`tracemalloc`), et lire des fichiers CSV (`csv`).
- **Structures de données prêtes à l'emploi** (listes dynamiques, dictionnaires), dont le fonctionnement interne et la complexité sont connus et seront expliqués.

Contrepartie assumée : Python est un langage interprété. Une même boucle s'exécute en général 10 à 100 fois plus lentement qu'en C. Cela ne change pas les complexités, mais cela change les temps mesurés. L'étude des performances en tiendra compte.

Respect du sujet (partie 9) : les algorithmes qui constituent le cœur du travail (tris, optimisation du chargement) sont **implémentés à la main**, et leur fonctionnement est expliqué dans ce rapport. Lorsqu'une fonction native est utilisée pour une opération courante, l'algorithme sous-jacent et sa complexité sont précisés. Aucune bibliothèque externe n'est utilisée.

2. Organisation du projet

2.1 Arborescence

Le dépôt respecte l'organisation demandée (dossier `src/`, `README.md`, `rapport.pdf`), complétée par trois dossiers utiles :

```
projet/
├── src/          code source du programme
├── tests/        tests unitaires
├── docs/         source de ce rapport et captures d'écran
├── outils/       script de génération du rapport PDF
├── README.md     présentation et mode d'emploi
└── rapport.pdf   ce rapport
```

2.2 Démarche de travail

Le projet a été réalisé par étapes successives. Chaque étape a été **validée** (tests unitaires et exécution du programme) avant de passer à la suivante, puis enregistrée par un commit sur le dépôt GitHub. L'historique des commits reflète donc la progression du travail.